# CreditShield: Loan Default Risk Prediction with Explainable AI

Final Report

23L-2524
23L-2560

**Deployment URL:**

`https://creditshield-loan-default-predictor.streamlit.app`

## Abstract

This project builds a machine learning application that predicts loan default risk using the Give Me Some Credit dataset (150K borrowers, 10 financial features). We engineered 3 additional features, trained an XGBoost classifier with SMOTE for class imbalance handling, and achieved an AUC-ROC of 0.85. The model classifies applicants into Low, Moderate, or High Risk categories, with 80% of actual defaulters correctly placed in the Moderate/High Risk groups. The application is deployed as a Streamlit web app featuring manual input, batch CSV processing, SHAP-based explainability, and a What-If analysis tool for real-time risk exploration.

## Contents

# 1 Phase 1: Problem Definition

## 1.1 Problem Statement

Financial institutions need fast, consistent, and explainable credit risk assessment. This project builds an ML-powered system that accepts applicant financial data and returns a default-risk prediction classified into Low, Moderate, or High Risk, along with SHAP-based explanations showing why.

## 1.2 Target Audience

Loan officers, credit analysts, and risk management teams at banks and lending institutions who need data-driven, auditable risk scoring.

## 1.3 AI Justification

Traditional rule-based scoring uses fixed thresholds that miss complex interactions between variables. ML models capture non-linear patterns (e.g., high debt ratio is acceptable for borrowers with zero delinquencies but dangerous with past late payments). XGBoost handles tabular data well, provides feature importance, and supports probability outputs needed for threshold tuning and risk categorization.

# 2 Phase 2: Data Acquisition & Preprocessing

## 2.1 Dataset

**Give Me Some Credit** from Kaggle — 150,000 borrowers with 10 financial features and a binary target (default within 2 years). Class imbalance: 93.3% No Default / 6.7% Default.

## 2.2 Preprocessing Pipeline

1. **Column mapping**: Renamed all columns for readability
2. **Missing values**: Median imputation for MonthlyIncome (19.8% missing) and Dependents (2.6% missing); Age = 0 replaced with median
3. **Duplicates**: 767 rows removed
4. **Outliers**: Sentinel codes (96, 98) in late payment columns capped at 20; RevolvingUtilization, DebtRatio, MonthlyIncome, OpenCreditLines, RealEstateLines capped at 99th percentile
5. **Scaling**: RobustScaler (uses median/IQR, robust to remaining skew)

## 2.3 Key EDA Findings

- Default rate decreases with age: 11.7% for under-30 vs 2.3% for 70+
- Default rate increases with credit utilization: 2.0% at 0–25% vs 37.2% at >100%
- Late payment features are highly correlated (0.70–0.84) with each other
- Strongest individual predictors of default: RevolvingUtilization (0.28), Times30_59Late (0.25), Times90Late (0.24)

# 3 Phase 3: Model Architecture

## 3.1 Pipeline Design

Feature Engineering → 70/15/15 Split → RobustScaler → SMOTE (0.7) → XGBoost → Feature Selection → Threshold Tuning

## 3.2 Feature Engineering

Three derived features were added:

| Feature | Formula | Purpose |
|---|---|---|
| TotalLatePayments | Sum of all late payment counts | Combines 3 weak signals into 1 |
| HasSevereDelinquency | 1 if Times90Late > 0 | Binary flag for severe risk |
| IncomeDebtRatio | Income / (DebtRatio + 1) | Higher = safer borrower |

## 3.3 Model Selection

We tested four models: Logistic Regression, Random Forest, Gradient Boosting, and XGBoost. XGBoost was selected as the final model for the following reasons:

- Handles tabular data well without extensive preprocessing
- Provides built-in feature importance for interpretability
- Supports probability outputs needed for threshold tuning and risk categorization
- Responds well to hyperparameter tuning, consistently outperforming other models after optimization

## 3.4 Feature Selection

After training with all 13 features, the top 7 by importance were selected. The engineered feature TotalLatePayments became the most important predictor, confirming that combining three weak signals produced a stronger feature than any individual column.

## 3.5 Hyperparameters

Final configuration: n_estimators = 200, max_depth = 5, learning_rate = 0.1, subsample = 0.8, SMOTE ratio = 0.7.

# 4 Phase 4: Evaluation

## 4.1 Validation Strategy

70/15/15 stratified Train/Validation/Test split. Training set used for SMOTE and model fitting. Validation set used for feature selection, threshold tuning, and comparison. Test set touched once for final evaluation. 5-Fold Stratified CV run separately for consistency.

### 4.2 Why These Metrics?

Accuracy is misleading at 93/7 imbalance — a model predicting "No Default" for everyone gets 93% accuracy but catches zero defaulters. We report Precision, Recall, F1, and AUC-ROC. For lending, Recall matters most: missing a defaulter costs more than a false alarm.

### 4.3 Final Results

| Set | Precision | Recall | F1 | AUC-ROC |
|---|---|---|---|---|
| Validation | 0.3455 | 0.5177 | 0.4144 | 0.8535 |
| Test | 0.3422 | 0.5127 | 0.4105 | 0.8509 |

Table 1: Validation vs Test — differences $< 0.01$ confirm no overfitting

### 4.4 Risk Categories

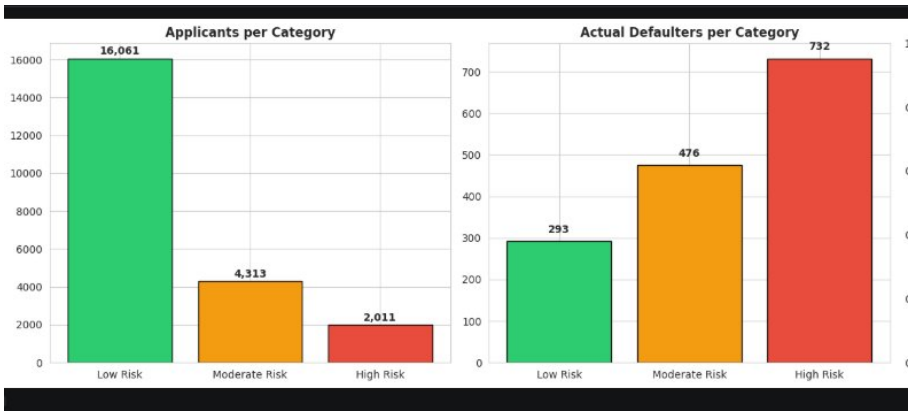


Figure 1: Applicants per risk category and actual defaulters per category

| Category | Applicants | Defaulters | % of Defaulters | Default Rate |
|---|---|---|---|---|
| Low Risk | 16,061 | 293 | 19.5% | 1.8% |
| Moderate Risk | 4,313 | 476 | 31.7% | 11.0% |
| High Risk | 2,011 | 732 | 48.8% | 36.4% |

Table 2: 80.5% of defaulters captured in Moderate + High Risk

### 4.5 Error Analysis

The model's false negatives (missed defaults) have significantly fewer late payments, lower utilization, and almost no severe delinquency. These "quiet defaulters" look safe on paper but still default, likely due to sudden life events that historical credit data cannot predict.

### 4.6 Bias Check

Performance varies by age group (better for younger borrowers where defaults are more common) but is consistent across income groups. Variation is driven by class distribution, not algorithmic bias. No protected attributes (race, gender) are used as features.

## 5 Phase 5: Deployment

### 5.1 Deployment Details

| Property | Value |
|---|---|
| Platform | Streamlit Community Cloud |
| URL | `https://creditshield-loan-default-predictor.streamlit.app` |
| Framework | Streamlit (Python) |
| Model | XGBoost (loaded from pickle) |
| Explainability | SHAP (TreeExplainer) |

The application provides three modes of interaction through a tabbed interface: Manual Input for individual applicants, CSV Upload for batch processing, and What-If Analysis for real-time risk exploration.

### 5.2 Tab 1: Manual Input

The user enters 10 financial attributes for an individual applicant and clicks "Predict Risk." The app runs the full preprocessing pipeline (outlier capping, feature engineering, scaling) and returns:

- The risk category (Low, Moderate, or High Risk) with a colored banner
- The exact default probability as a percentage
- A SHAP bar chart showing which features drove this specific prediction
- A text summary of the top 3 contributing factors

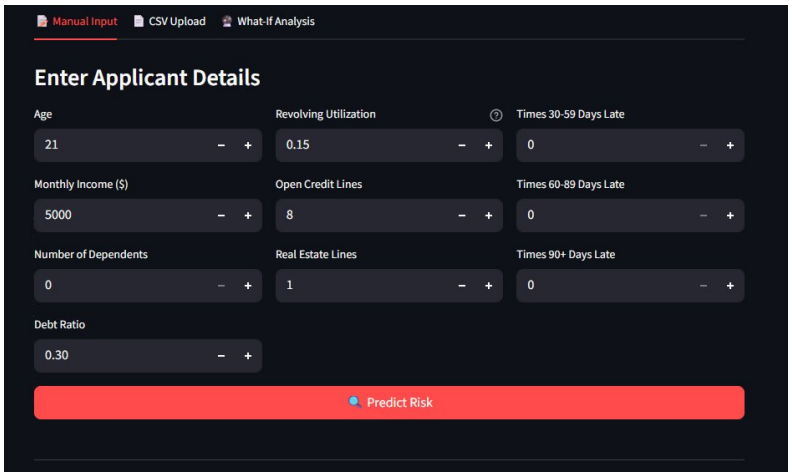


Figure 2: Manual input form with 10 applicant fields

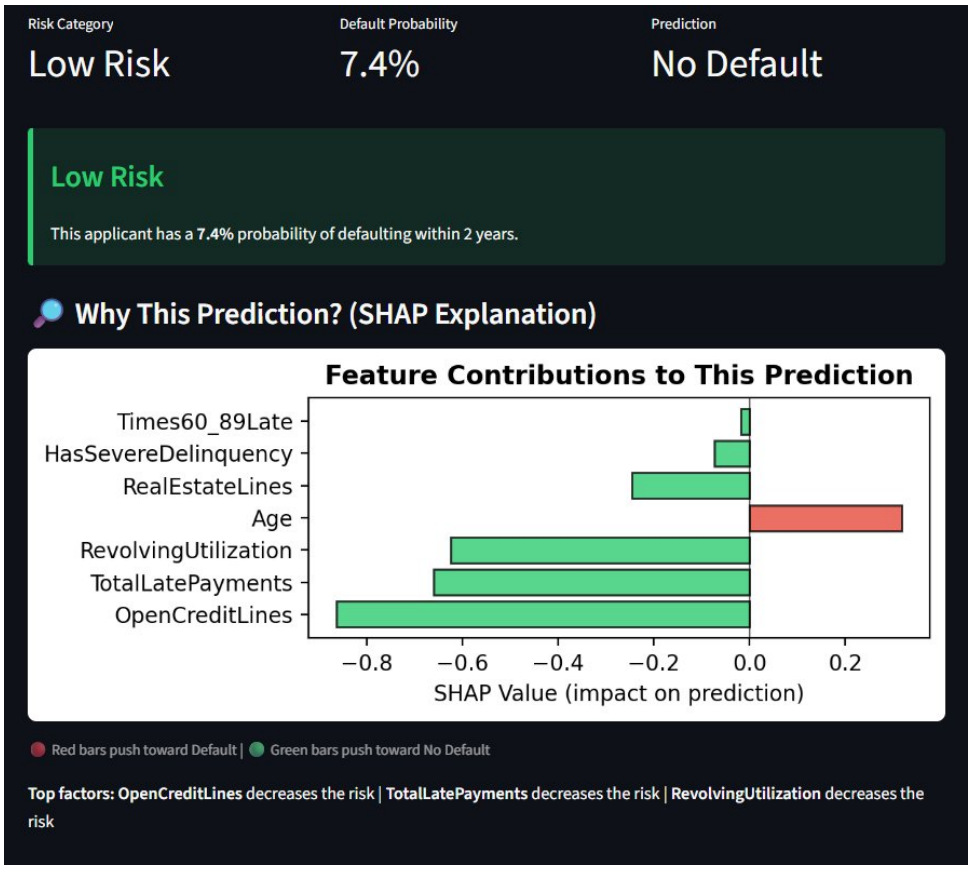


Figure 3: Prediction result with SHAP explanation — this applicant is classified as Low Risk (7.4% default probability). The SHAP chart shows OpenCreditLines and TotalLatePayments are the strongest factors reducing risk, while Age is the only factor slightly increasing it.

### 5.3 Tab 2: CSV Upload

The user uploads a CSV file containing multiple applicants. The app auto-detects both the original Kaggle column names and the mapped names, preprocesses all rows using training-set parameters, and returns batch predictions.

We tested this by uploading the Kaggle competition's test file (cs-test.csv, 101,503 applicants) which has no target column — exactly the scenario the app is designed for.

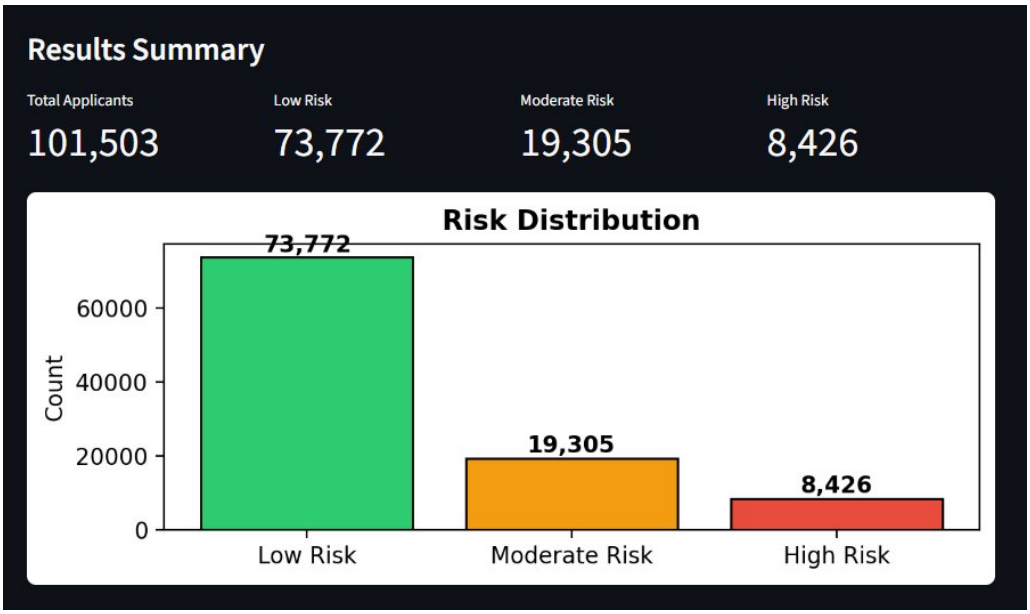


Figure 4: Batch prediction results on 101,503 applicants from the Kaggle test set. The model classified 73,772 as Low Risk (72.7%), 19,305 as Moderate Risk (19.0%), and 8,426 as High Risk (8.3%). Results are downloadable as a CSV file.

### 5.4 Tab 3: What-If Analysis

This tab satisfies the requirement for an interactive "What-If" feature. All 10 input variables are presented as sliders. As the user adjusts any slider, the risk score, risk category, and SHAP explanation chart update in real time.

This allows a loan officer to explore questions like: "If this applicant's income increased from $3,000 to $6,000, would they still be High Risk?" or "How many late payments does it take to push someone from Low to Moderate Risk?"


Figure 5: What-If Analysis sliders — the user can adjust any variable in real time

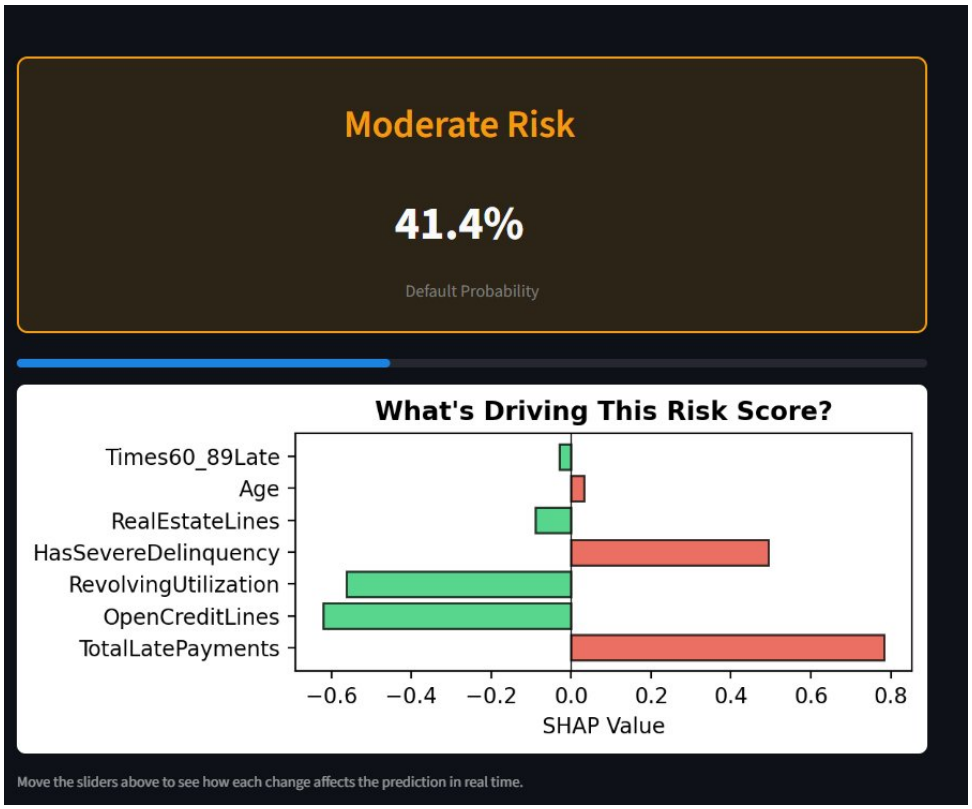


Figure 6: What-If result for a 25-year-old with 1 late payment at 90+ days: Moderate Risk at 41.4%. The SHAP chart shows TotalLatePayments and HasSevereDelinquency are the primary risk drivers, while OpenCreditLines and RevolvingUtilization are reducing the risk.

### 5.5 Explainability (SHAP)

Every prediction across all three tabs includes a SHAP explanation. SHAP (SHapley Additive exPlanations) computes the contribution of each feature to a specific prediction. Red bars indicate features pushing toward default, green bars push toward no default. The bar length represents the magnitude of impact.

This satisfies the XAI requirement: the user does not just see "High Risk" — they see *why* the model made that decision and which factors they could address to reduce the risk.

## 6 Conclusion

### 6.1 Summary

CreditShield successfully demonstrates an end-to-end ML pipeline: from raw data preprocessing through model training to a deployed, explainable web application. The XGBoost model achieves an AUC-ROC of 0.85, correctly placing 80.5% of actual defaulters into Moderate or High Risk categories. High Risk applicants have a 36.4% default rate compared to 1.8% for Low Risk — a 20x separation.

### 6.2 Limitations

- 19.5% of defaulters still fall into Low Risk ("quiet defaulters" with clean-looking profiles)
- The model relies entirely on historical financial data and cannot predict defaults caused by sudden life events
- The dataset lacks non-financial features (employment type, loan purpose) that could improve predictions

### 6.3 Future Work

- Incorporate additional data sources (employment verification, loan purpose, collateral)
- Experiment with deep learning models or ensemble stacking
- Add time-series credit behavior features (trend of utilization over months)

## 7 Repository Structure

```
CreditShield-Loan-Default-Risk-Predictor/
|-- app/
|   |-- app.py
|   |-- models/
|       |-- best_model.pkl
|       |-- best_threshold.pkl
|       |-- feature_cols.pkl
|       |-- final_scaler.pkl
|       |-- outlier_caps.pkl
|       |-- train_medians.pkl
|-- data_scripts/
|   |-- preprocess.py
|   |-- eda.py
|   |-- outliers_handling.py
|   |-- scaling_normalization.py
|-- figures/
|-- outputs/
|-- reports/
|-- .gitignore
|-- README.md
|-- requirements.txt
```